

操作系统, 2026

第10讲 存储器管理(3/3)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

Windows的内存管理

02

Linux的内存管理

Windows的内存管理

- 内存管理涉及的实体
- 虚拟地址空间布局
- 地址转换机制
- 缺页处理
- 内存分配方式
- 工作集
- 物理内存管理

针对Windows 11, 64位x86-64硬件平台

内存管理涉及的实体

■ 虚拟内存

- 程序员看到的地址空间，还没有被MMU转换

■ 物理内存

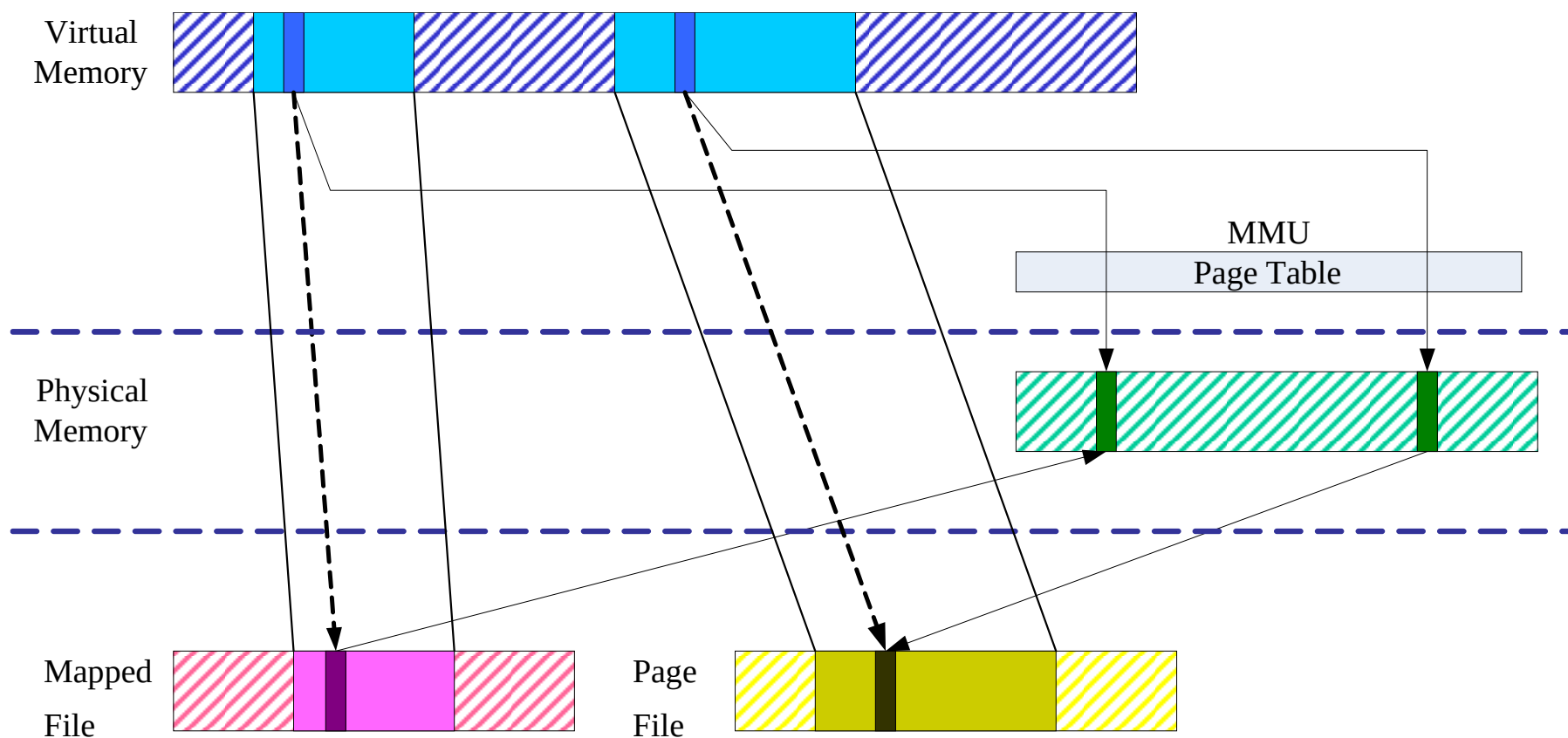
- 系统中实际安装的内存
- 通过物理地址或实地址访问

■ 页文件或内存映射文件

- 磁盘上的文件
- 作为虚拟内存的后备存储

内存管理涉及的实体

■ 关系



虚拟地址空间布局

- 理论上64位地址空间是 2^{64} B（16EB），但实际硬件（x86-64）并没有用满：
 - 现代 CPU（如 x86-64）通常支持48位虚拟地址
 - Windows 11使用的是规范地址(canonical address)
- 规范地址
 - 低48位有效，第47位是最高有效位（MSB）
 - 高16位必须是符号扩展（第47位的复制）

虚拟地址空间布局

- 稀疏性：地址空间非常大，但绝大多数未映射

0xFFFFFFFF`FFFFFFFF

0xFFFF8000`00000000

0x00007FFF`FFFFFFFF

0x00000000`00000000

高地址区（内核态）

未使用/无效区（防止越界访问）

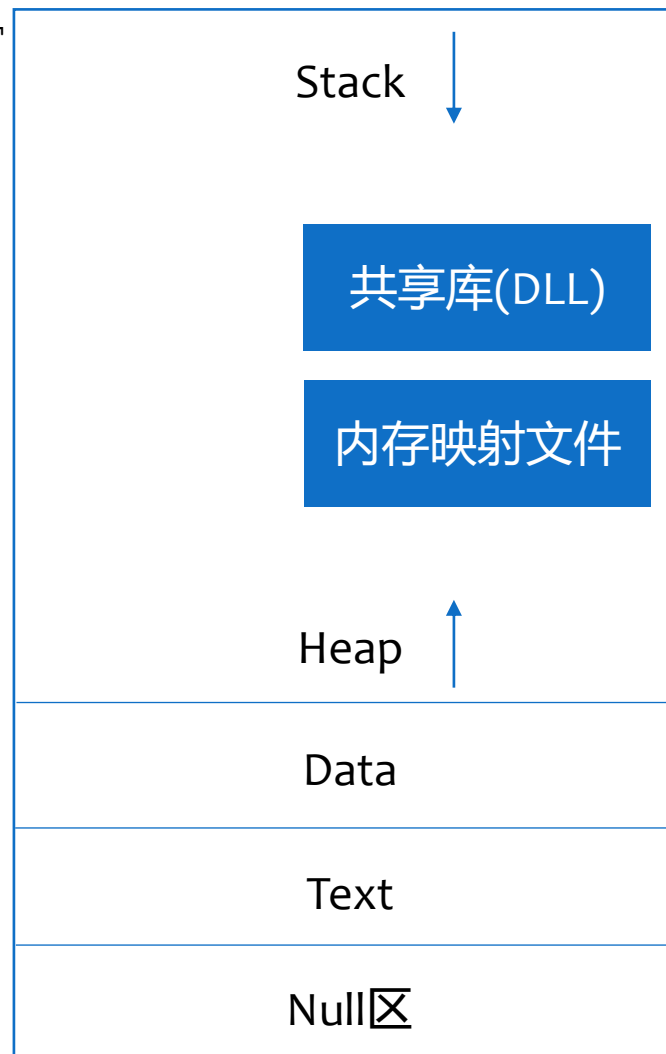
低地址区（用户态）

用户空间地址布局（概念性）

- 128TB
- 每个进程独立拥有
- 互相隔离（页表不同）

0x00007FFF`FFFFFFFF

0x00000000`00000000



防止空指针

内核空间地址布局（概念性）

0xFFFFFFFF`FFFFFFFF

- 128TB
- 所有进程共享，页表中这一部分是相同映射
- 内核地址空间本质上是一个稀疏、动态、部分受约束的地址区间集合，而不是按功能顺序排好的线性布局

0xFFFF8000`00000000

内核代码

内核数据

分页池
(Paged Pool)

非分页池
(Nonpaged Pool)

系统页表 (PTE 区)

设备映射 (MMIO)

虚拟地址空间布局

- Windows 11在地址布局中嵌入了大量安全机制
- ASLR (Address Space Layout Randomization, 地址空间布局随机化) : DLL、堆、栈位置随机, 防止利用固定地址攻击
- DEP (Data Execution Prevention, 数据执行保护) : 页有 NX (No Execute) 位, 数据页不能执行代码
- KASLR (Kernel Address Space Layout Randomization, 内核地址空间布局随机化) : 内核各部分随机化, 打破固定结构

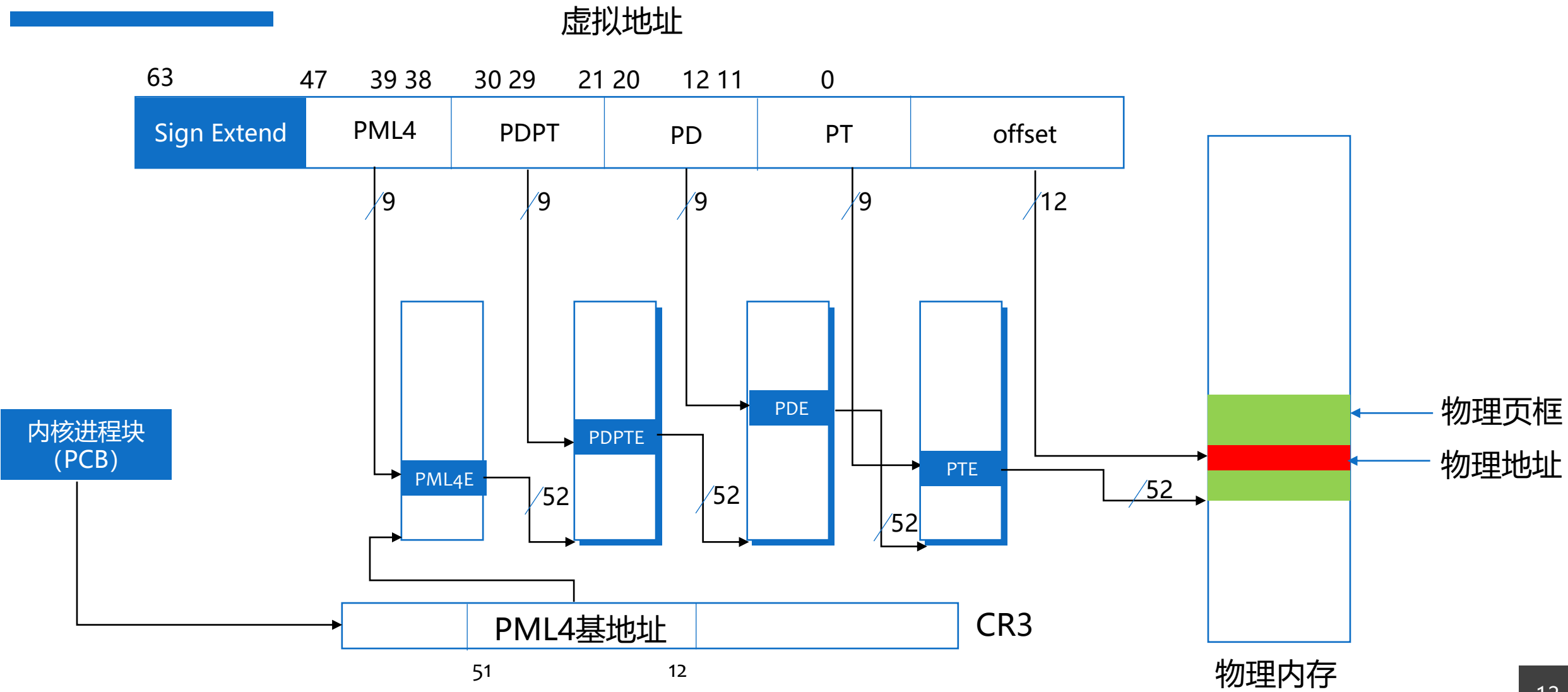
地址转换机制

- Windows 11使用四级页表，每一级索引9位
 - PML4 (Page Map Level 4) : 第4级页映射
 - PDPT (Page Directory Pointer Table) : 页目录指针表
 - PD (Page Directory) : 页目录
 - PT (Page Table) : 页表
- 每页大小4KB
- 支持大页



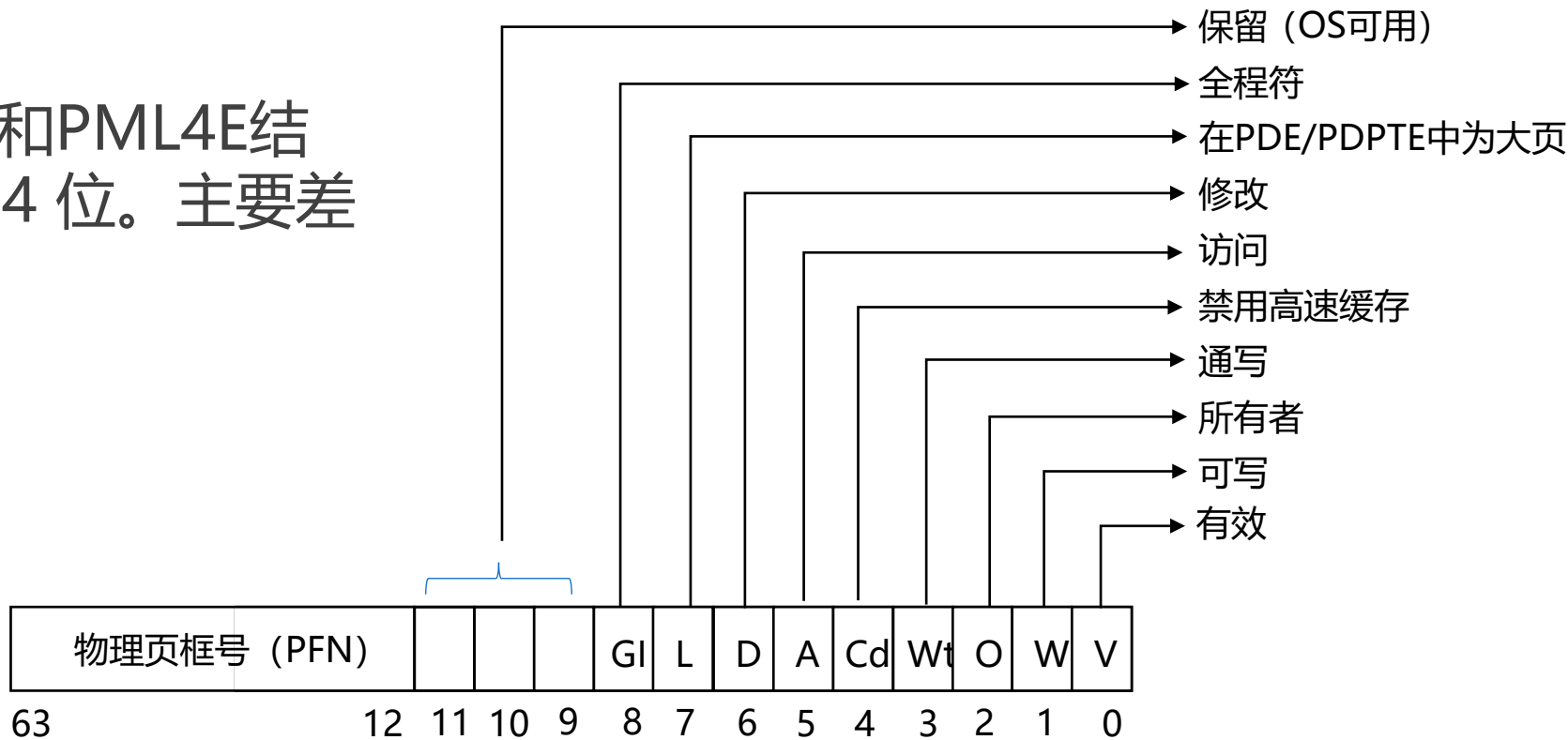
地址转换机制

PTE (Page Table Entry)
PDE (Page Directory Entry)
PDPTE (Page Directory Pointer Table Entry)
PML4E (Page Map Level 4 Entry)



地址转换机制

- 页表项格式
- PTE、PDE、PDPTE和PML4E结构基本相同，都是 64 位。主要差别在L位



大页支持

- x86-64 (Windows 11所运行的硬件基础) 支持大页 (Large Pages) 和超大页 (Huge Pages), 作为对标准4KB页的扩展
- 大页和超大页是用更少的页表项, 映射更大的一段连续内存
- 用更大的粒度做地址映射, 以换取性能和管理开销上的收益

类型	单页大小	覆盖范围
普通页	4 KB	细粒度
大页	2 MB	512×4KB
超大页	1 GB	512×2MB

大页支持

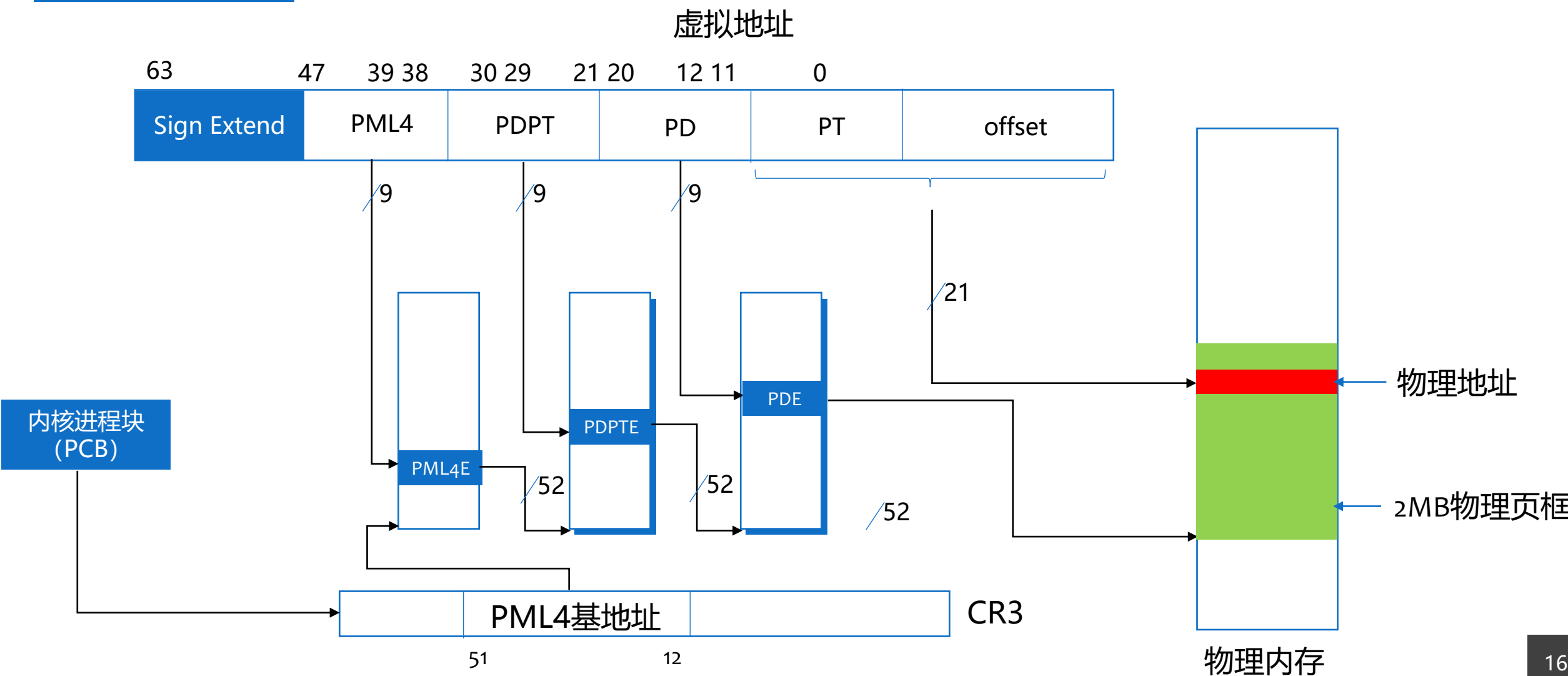
- 支持大页的关键是提前截断PML4 → PDPT → PD → PT → 4KB页链条
- 2MB大页：在PDE层设置L=1，不再需要PT层

PML4 → PDPT → PDE (L=1) → 2MB 物理页

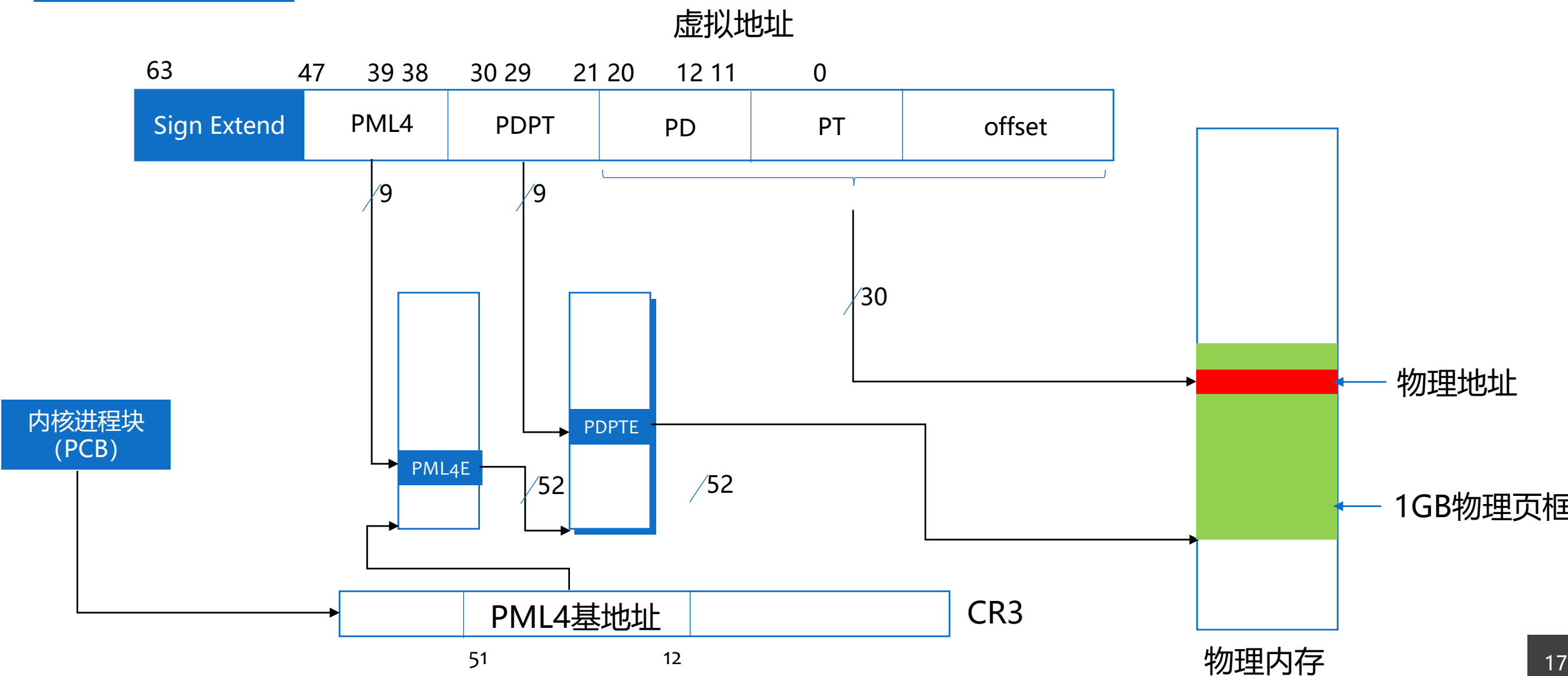
- 1GB超大页：在PDPTE层设置L=1，不再需要PD和PT层

PML4 → PDPTE (L=1) → 1GB 物理页

大页支持——2MB大页



大页支持——1GB大页



大页支持

■大页的优点

- 减少TLB压力，页越大，TLB命中率越高。例如，映射 1GB 内存

页大小	需要的页数	TLB 项数
普通页	262,144	很多
大页	512	少
超大页	1	极少

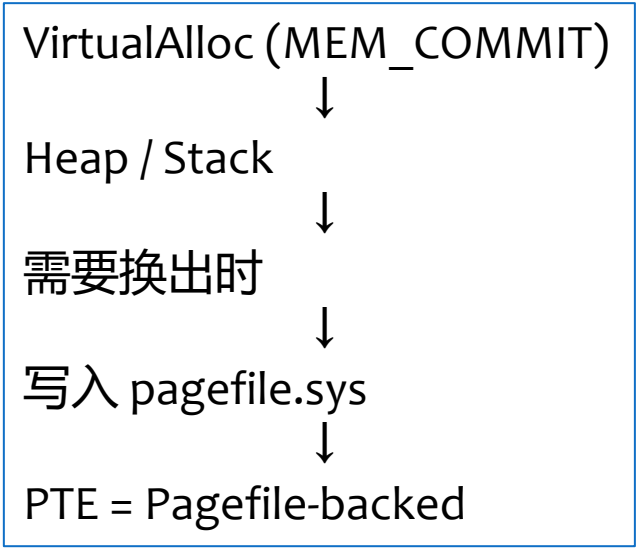
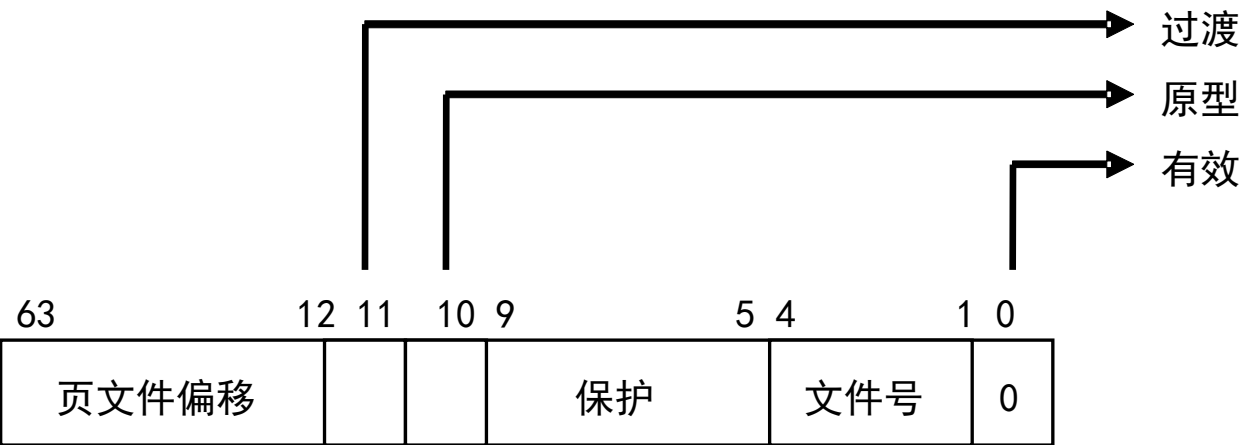
- 减少页表开销：更少的页表项，更少的页表遍历
- 提高连续访问性能，有利于大数组、数据库、科学计算、虚拟化
- 大页的缺点：内碎片大、页面替换开销大

缺页处理

- 页目录项和页表项的最低位为1，有效(Valid)，表示该页映射了物理内存
- 当要访问的虚拟地址所在页在物理内存中时，该虚拟地址所在页相应的PML4E、PDPTE、PDE，PTE都有效，CPU的MMU自动根据相应的PML4E、PDPTE、PDE，PTE把虚拟地址转换成物理地址，完成访问——这一过程操作系统不需介入
- 如果要访问的虚拟地址所在页在不物理内存中，此时的PTE无效(最低位为0)。对无效PTE的格式的定义，由操作系统负责

无效的页表项——软件PTE

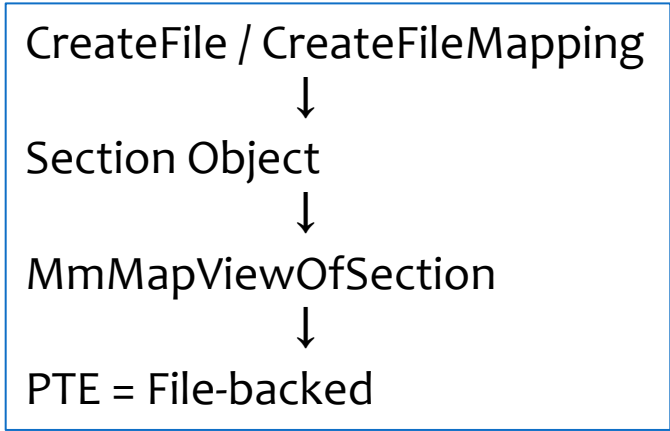
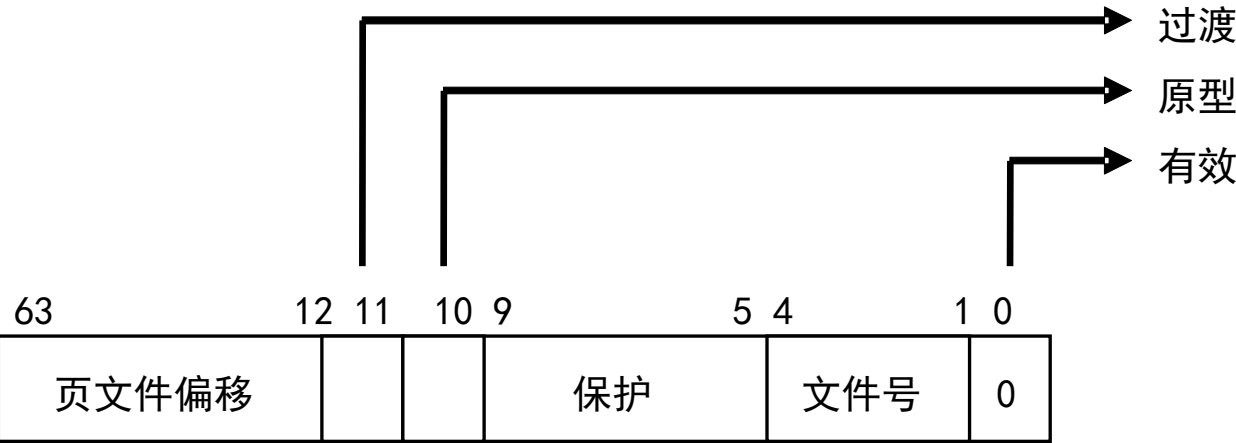
- 页文件 (Pagefile-backed)
 - 所需的页驻留在一个页文件 (pagefile.sys) 中，引发页面调入操作
 - 由MMPTE_SOFTWARE结构定义



无效的页表项——软件PTE

■ 内存映射文件

- 来源为匿名内存（anonymous memory），例如Heap和Stack
- 利用原型和过渡位的组合，通过原型PTE间接定位Section Object，再到File Object，最终定位到内存映射文件



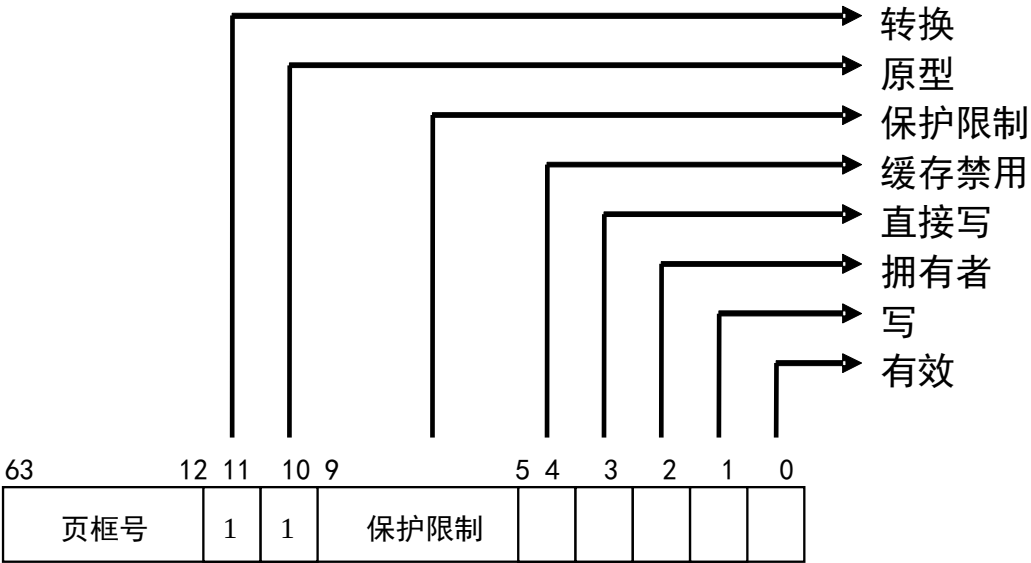
无效的页表项——软件PTE

■ 过渡

- 所需页面在内存中的后备链表、修改链表或修改尚未写入链表
- 由MMPTE_TRANSITION结构定义

■ 未知

- 页表项为零，或者页表不存在



缺页中断

- 当某条指令访问的页不在物理内存中时，CPU仍然会自动通过页目录和页表把虚拟地址转换成物理地址，在地址转换过程中，CPU将会发现对应地址的页表项无效，从而就会引发[页面错误 \(Page Fault\)异常](#)，该异常的中断号是0xe，有时也称为[缺页中断](#)
- 发生缺页中断时，CPU自动将引发异常时访问的虚拟地址存入寄存器CR2

缺页中断

- 在发生异常时，CPU自动根据中断号，在中断描述符表中找到相应的中断描述符，根据中断描述符中的地址，转到异常处理程序`ntoskrnl!KiTrap0E`
- 异常处理程序`KiTrap0E`是操作系统提供的，它将会调用`MmAccessFault`，`MmAccessFault`通过`CR2`中的访问地址，计算出相应的`PDE/PTE`地址，通过分析`PTE`中的内容(无效的`PDE/PTE`的格式由操作系统定义)，可以知道是哪种情况引起的异常，并根据情况作出相应的处理

页面调入I/O

- 向文件(页文件或映射文件)发出读操作来解决缺页问题
- 页面调入I/O是同步的

页文件

- 作为虚拟内存的后备存储空间
- 最多16个页文件
- 页文件以非压缩的形式被创建

用户空间内存分配方式

- 以页为单位的虚拟内存分配
 - Virtualxxx
- 通过内存映射文件提供大数据流和内存共享服务
 - CreateFileMapping, MapViewOfFile
- 内存堆方法
 - Heapxxx和早期的接口Localxxx和Globalxxx

以页为单位的虚拟内存分配

- 适合于管理大型对象数据结构，尤其是动态或稀疏分配的。对于以页为单位的虚拟内存，Windows采用两阶段内存分配方法：保留内存和提交内存
- 应用程序可以首先保留地址空间，然后向此地址空间提交物理页面。VirtualAlloc函数实现这些功能
- 保留地址空间是为线程将来使用所保留的一块虚拟地址。在已保留的区域中，提交页面必须指出将物理存储器提交到何处以及提交多少。提交页面在访问时会转变为物理内存中的有效页面
- VirtualFree函数回收页面或释放地址空间

以页为单位的虚拟内存分配

- 使用VirtualAlloc可以在用户地址空间中保留或者提交指定地址和大小的一段地址空间。那么系统如何知道指定的这段地址空间是不是已经被分配(保留或者提交)。对于指定地址空间是否已经被提交了物理内存，可以通过页目录和页表来判断，不过这样做很麻烦。而对于指定地址空间是否已经被保留，通过页目录和页表没有办法判断
- Windows中使用VAD来解决这个问题

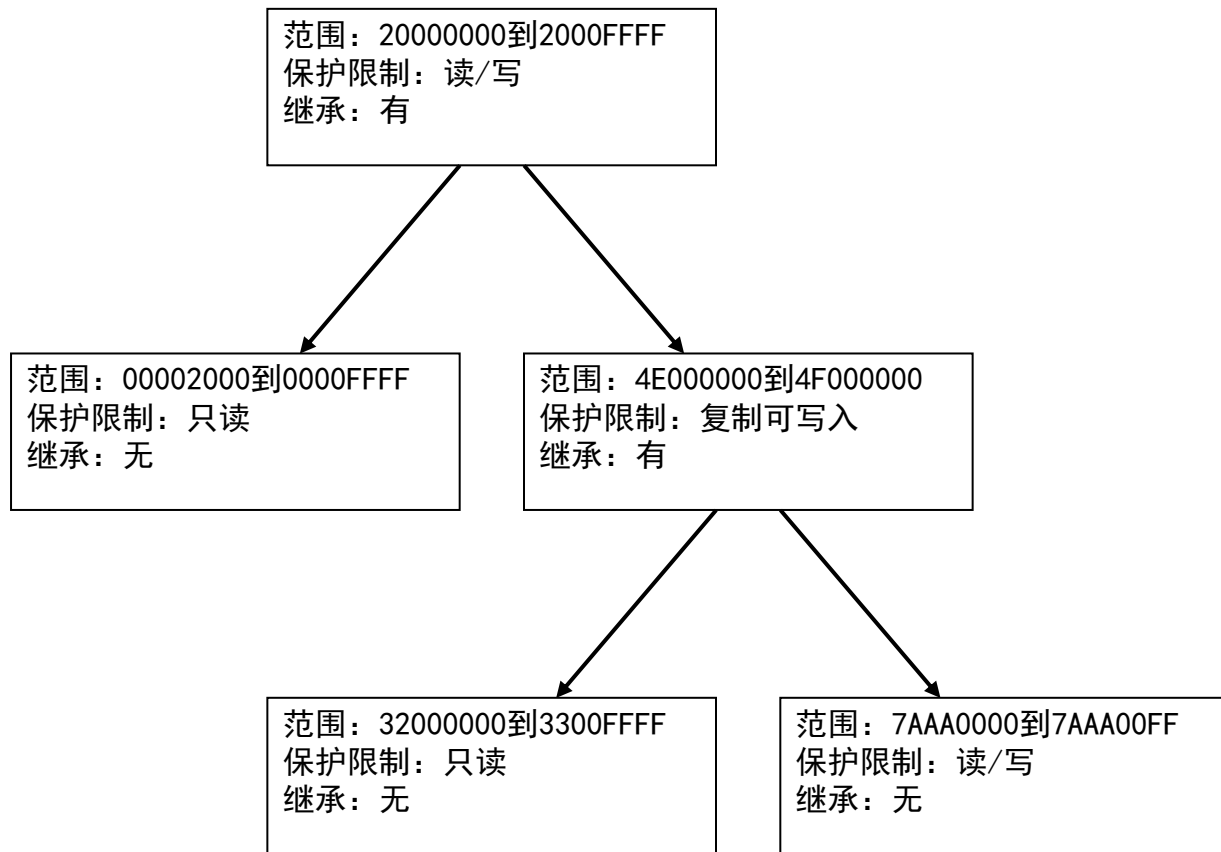
以页为单位的虚拟内存分配

■虚拟地址描述符(VAD)

- 对于每一个进程，Windows的内存管理器维护一组虚拟地址描述符(VAD, Virtue Address Descriptors)来描述一段被分配的进程虚拟空间的状态
- 虚拟地址描述信息被构造成一棵自平衡二叉树(AVL树)以使查找更有效率

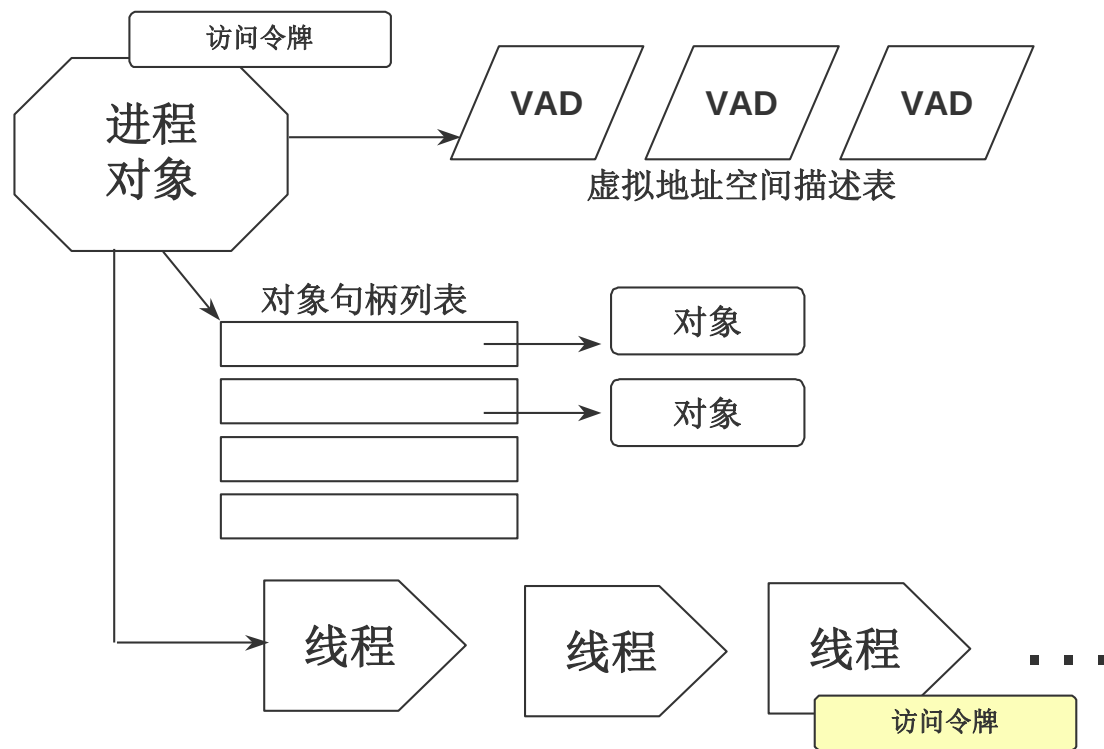
以页为单位的虚拟内存分配

虚拟地址描述符(VAD)



以页为单位的虚拟内存分配

- VAD二叉排序树的根的地址保存在进程结构EPROCESS中



内存映射文件

- 内存映射文件可以用来保留一个地址空间的区域，并作为该区域的后备存储
- 内存映射文件后备存储器来自磁盘上的普通文件，而不是系统的页文件
- 一旦一个文件作为内存映射文件被映射，就可以像访问内存空间一样访问它

内存映射文件的应用

- 加载和执行exe和dll文件，这可以节省页文件空间和应用程序启动所需的时间
- 访问磁盘上的数据文件，这可以减少文件I/O，并且不必对文件进行缓存
- 实现多个进程间的数据共享

EXE文件和DLL文件

■ Windows创建一个进程时，将执行下列操作步骤：

1. 创建一个新进程执行体对象（即PCB）
2. 为新进程创建一个私有地址空间
3. 保留一个足够大的地址空间区域，用于该EXE文件。该区域的位置在EXE文件本身中设定。按照默认设置，EXE文件的基地址默认值是0x00000001`40000000，但是可以在创建应用程序的EXE文件时重载该地址，方法是在链接应用程序时使用链接程序的/BASE选项
4. 系统注意到支持已保留区域的物理存储器是在磁盘上的EXE文件中，而不是在系统的页文件中
5. 当EXE文件被映射到进程的地址空间中之后，系统将访问EXE文件的.idata节，它列出了包含在EXE文件中的代码要调用的函数的DLL文件。然后，系统为每个DLL文件调用LoadLibrary函数，如果任何一个DLL需要更多的DLL，那么系统将调用LoadLibrary函数，以便加载这些DLL。每当调用LoadLibrary来加载一个DLL时，系统将执行类似上述第3和第4个步骤

EXE文件和DLL文件

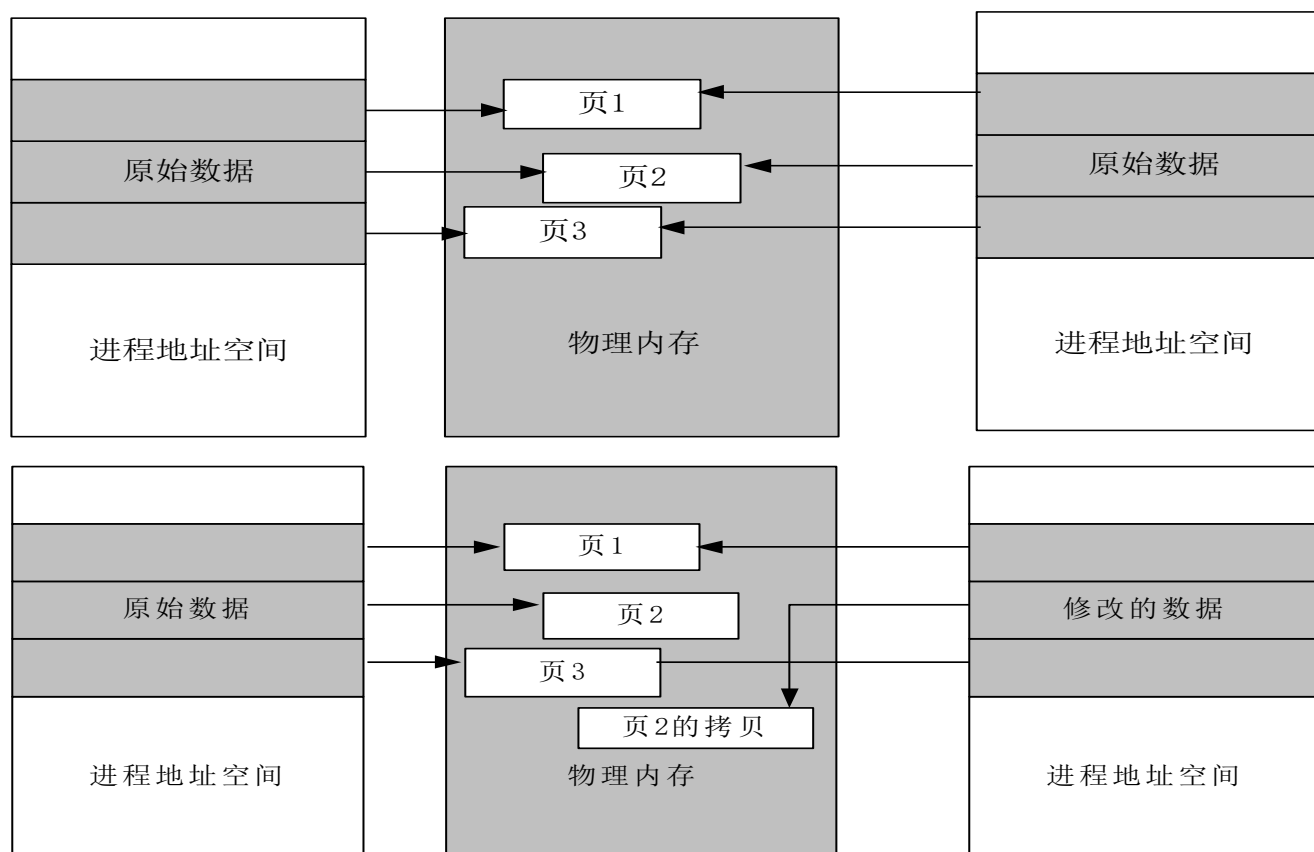
- 当为正在运行的应用程序创建新进程时，Windows将打开用于标识可执行文件映像的文件映射对象的另一个内存映射视图，并创建一个新进程对象。系统将新的进程ID赋予该对象
- 通过使用内存映射文件，同一个应用程序的多个正在运行的实例能够共享内存中的相同代码和数据

EXE文件和DLL文件

- 如果应用程序的一个实例改变了驻留在数据页面中的某些全局变量，那么该应用程序的所有实例的内存内容都将改变，这种改变可能带来灾难性的后果，因此是决不允许的
- 为此，Windows的内存管理器利用**写时复制(copy-on-write)**特性来防止进行这种改变。每当应用程序尝试将数据写入它的内存映射文件时，Windows就会抓住这种尝试，为包含应用程序尝试写入数据的内存页面分配一个新内存页，再复制该页面的内容，并允许该应用程序将数据写入这个新分配的内存页。结果，同一个应用程序的所有其他实例的运行都不会受到影响

EXE文件和DLL文件

■ 写时复制



内存映射数据文件

■问题：如何实现将一个文件中的所有字节倒放？

- 方法1：一个文件，一个缓冲区

- 方法2：两个文件，一个缓冲区

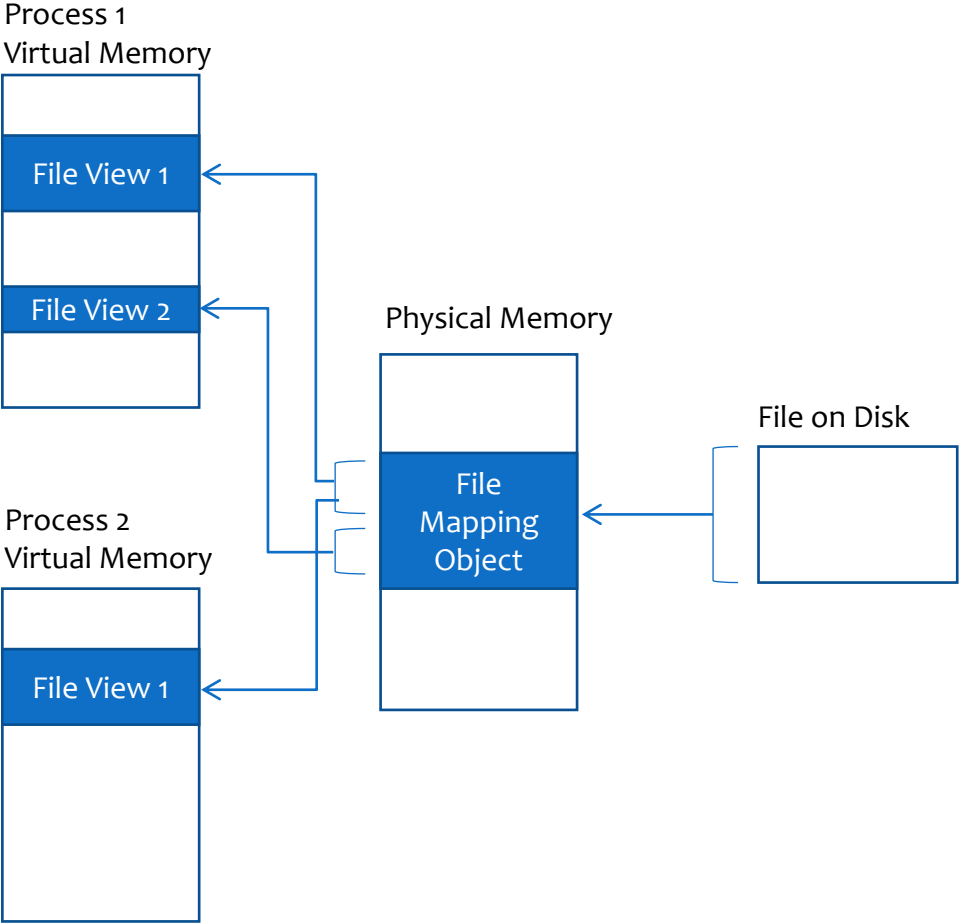
- 方法3：一个文件，两个缓冲区

- 方法4（内存映射）：一个文件，零缓冲区——最大优点是，操作系统能够管理所有的文件缓冲区操作，不必分配任何内存，或者将文件数据加载到内存，也不必将数据重新写入该文件，或者释放任何内存块

使用内存映射文件在进程之间共享数据

- 通过让两个或多个进程映射同一个文件映射对象的视图，可以实现数据共享，因为它们将共享物理存储器的同一个页面。因此，当一个进程将数据写入一个共享文件映射对象的视图时，其他进程可以立即看到它们视图中的数据变更情况
- 注意，如果多个进程共享单个文件映射对象，那么所有进程必须使用相同的名字来表示该文件映射对象

使用内存映射文件在进程之间共享数据



内存映射文件的使用过程

- 若要使用内存映射文件，必须执行下列操作步骤：
 1. 创建或打开一个文件对象，该对象用于标识磁盘上想用作内存映射文件的文件
 2. 创建一个文件映射对象，告诉系统该文件的大小和打算如何访问该文件
 3. 让系统将文件映射对象的全部或一部分映射到进程地址空间中

内存映射文件的使用过程

- 当完成对内存映射文件的使用时，必须执行下面这些步骤将它清除：
 1. 告诉系统从进程的地址空间中撤消文件映射对象的映像
 2. 关闭文件映射对象
 3. 关闭文件对象

内存映射文件的使用过程

■ 实例

```
#include <windows.h>
#include <stdio.h>

void main()
{
    HANDLE hFile, hFileMap;
    char *cBuffer;
    int dwFileSize, dwFileSizeHigh;
    int i;

    hFile = CreateFile(TEXT("testfilemap.c"),
                      GENERIC_READ,           // open for reading
                      FILE_SHARE_READ,        // share for reading
                      NULL,                    // default security
                      OPEN_EXISTING,           // existing file only
                      FILE_ATTRIBUTE_NORMAL,   // normal file
                      NULL);                   // no attr. template
```

内存映射文件的使用过程

■ 实例

```
if (hFile == INVALID_HANDLE_VALUE)
{
    printf("Could not open file (error %d)\n", GetLastError());
    return;
}

dwFileSize = GetFileSize(hFile, &dwFileSizeHigh);

hFileMap = CreateFileMapping(hFile,
                            NULL,                // default security
                            PAGE_READONLY,         // readonly access
                            0,                     // max. object size
                            dwFileSize,           // buffer size
                            TEXT("SECTION_TEST")); // name of mapping object

if (hFileMap == NULL || hFileMap == INVALID_HANDLE_VALUE)
{
    printf("Could not create file mapping object (%d).\n", GetLastError());
    return;
}
```

内存映射文件的使用过程

■ 实例

```
cBuffer = MapViewOfFile(hFileMap,
                        FILE_MAP_READ,           // read only
                        0,
                        0,
                        dwFileSize);

printf("Buffer = %x\n", cBuffer);

getch();
for(i = 0; i < dwFileSize; i++)
{
    printf("%c", cBuffer[i]);
}
getch();

if (!UnmapViewOfFile(cBuffer))
{
    printf("Could not unmap view of file (%d).\n", GetLastError());
    return;
}

CloseHandle(hFile);
CloseHandle(hFileMap);
}
```

内存堆方法

- 内存堆方法适合于大量的小型内存申请。堆(heap)是保留的地址空间中一个或多个页组成的区域，这个地址区域可以由堆管理器按更小块划分和分配。堆管理器是执行体中分配和回收可变内存的函数集
- 进程启动时带有一个缺省进程堆，通常是1MB大小。为了从缺省堆中分配内存，线程必须调用GetProcessHeap函数得到一个指向它的句柄。有了句柄后，线程可以调用HeapAlloc和HeapFree来从堆中分配和回收内存块
- 进程也可以使用HeapCreate函数创建另外的私有堆。当进程不再需要私有堆时，可以通过调用HeapDestroy释放虚拟地址空间

系统内存分配

■非分页缓冲池

- 由系统虚拟地址组成，它们长期驻留在物理内存中，在任何时候都可以被访问到(从任何IRQL级和任何进程上下文)，而不会发生页错误

■分页缓冲池

- 系统可以被分页和换出的虚拟内存的一个区域。不从DPC/调度级或更高级访问内存的设备驱动程序可以使用分页缓冲池

工作集

- 进程工作集：为每个进程分配的一定数量的页框
- 系统工作集：为可分页的系统代码和数据分配的页框

页面调度策略

- 取页策略：内存管理器利用请求式页面调度算法以及簇(集群)方式将页面装入内存
- 置页策略：选择页框应使CPU内存高速缓存不必要的震荡最小
- 换页策略
 - 在多处理器系统中，Windows采用了局部先进先出置换策略
 - 在单处理器系统中，采用最近最少使用策略(LRU)

系统工作集的内容

- 系统高速缓存页面
- 分页缓冲池
- Ntoskrnl.exe中可分页的代码和数据
- 设备驱动程序中可分页的代码和数据
- 系统映射视图(例如Win32k.sys)

物理内存管理

- 当操作系统需要分配一个物理页框给应用程序，来满足应用程序要求的时候，将遇到一个问题，系统如何知道哪些物理页框已经被使用，哪些物理页框没有被使用
- Windows使用页框号数据库(Page Frame Number DataBase)和相关结构用来解决这一问题
- Page Frame Number——PFN

物理内存管理

- 对于每一个物理页，操作系统使用一个24字节长的结构MMPFN来保存它的相关信息。页框号数据库就是一个MMPFN数组，这个数组的每一项对应一个物理页，例如数组第0项，对应物理页0，也就是页框号为0的物理页框
- Windows把页框号数据库的首地址保存在全局变量MmPfnDatabase中

物理内存中每个页面的状态

- 活动(Active)/有效(Valid)

- 这个物理页在某个进程的Working Set中，该进程的一个有效的页表项中的高20bit正是这个物理页的PFN

- 过渡(Transition)

- 系统正在从一个文件将内容读入该物理页，或者正在向一个文件写出该物理页内容

物理内存中每个页面的状态

■ 后备(stand by)

- 这个物理页曾经在某个进程的Working Set中，并且物理页中的内容在被该进程使用时没有被改变过，但是现在已经被移出该进程的Working Set。然而，物理页中的内容仍是在该进程Working Set中时的内容，该进程相应的PTE中的高20bit仍然是这个物理页的页框号，只是该PTE被标为invalid和transition
- 当该进程需要再次访问这一页的内容时，只需要重新设定该PTE的标志，并把该PTE变为有效。把

■ 修改不写入(Modified no-write)

- 内存管理器的Modified Page Writer将不会把这种物理页写入硬盘，其他和Modified物理页一样该物理页从Stand by状态变为Active(Valid)状态就可以了

物理内存中每个页面的状态

- 空闲(Free)

- 该物理页中的内容不再被需要

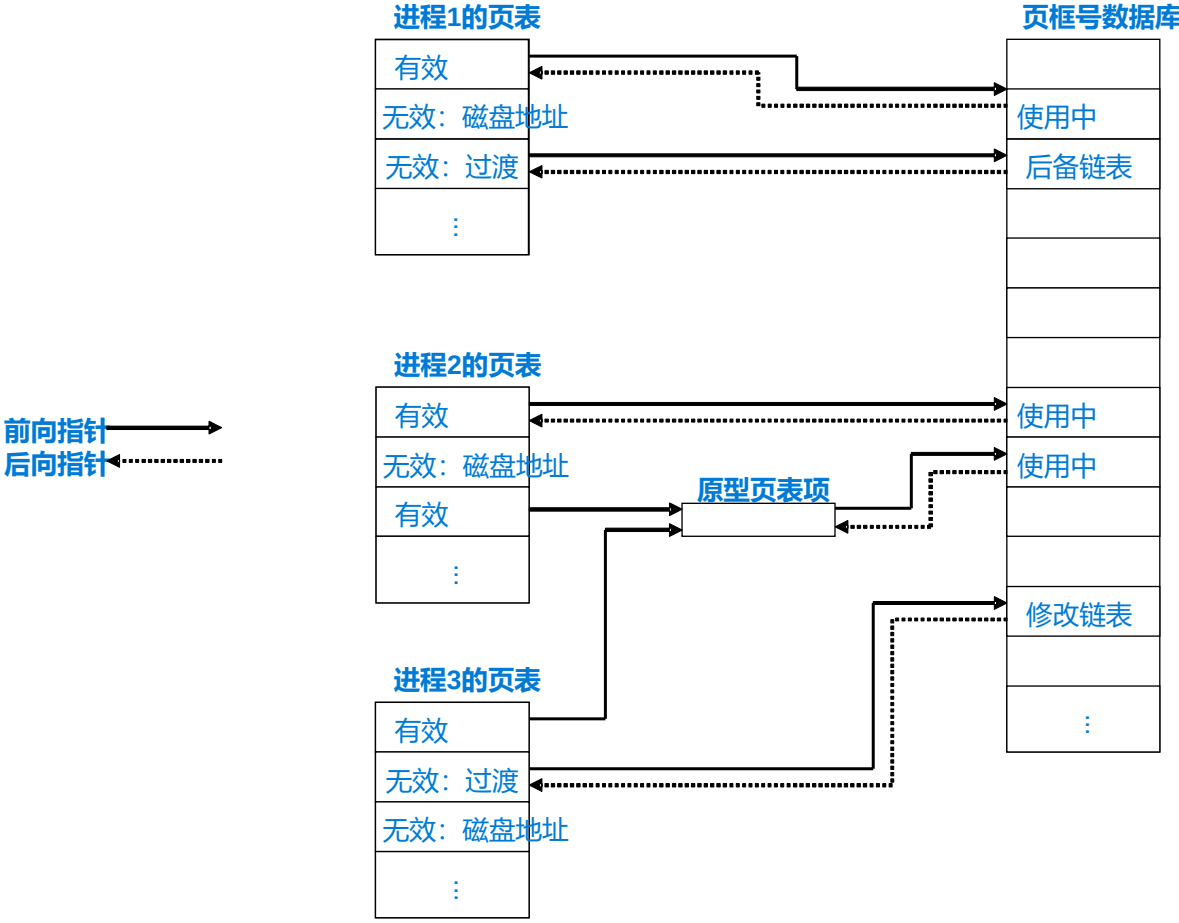
- 零初始化(zeroed)

- 该页free并且已经被用零初始化

- 坏(Bad)

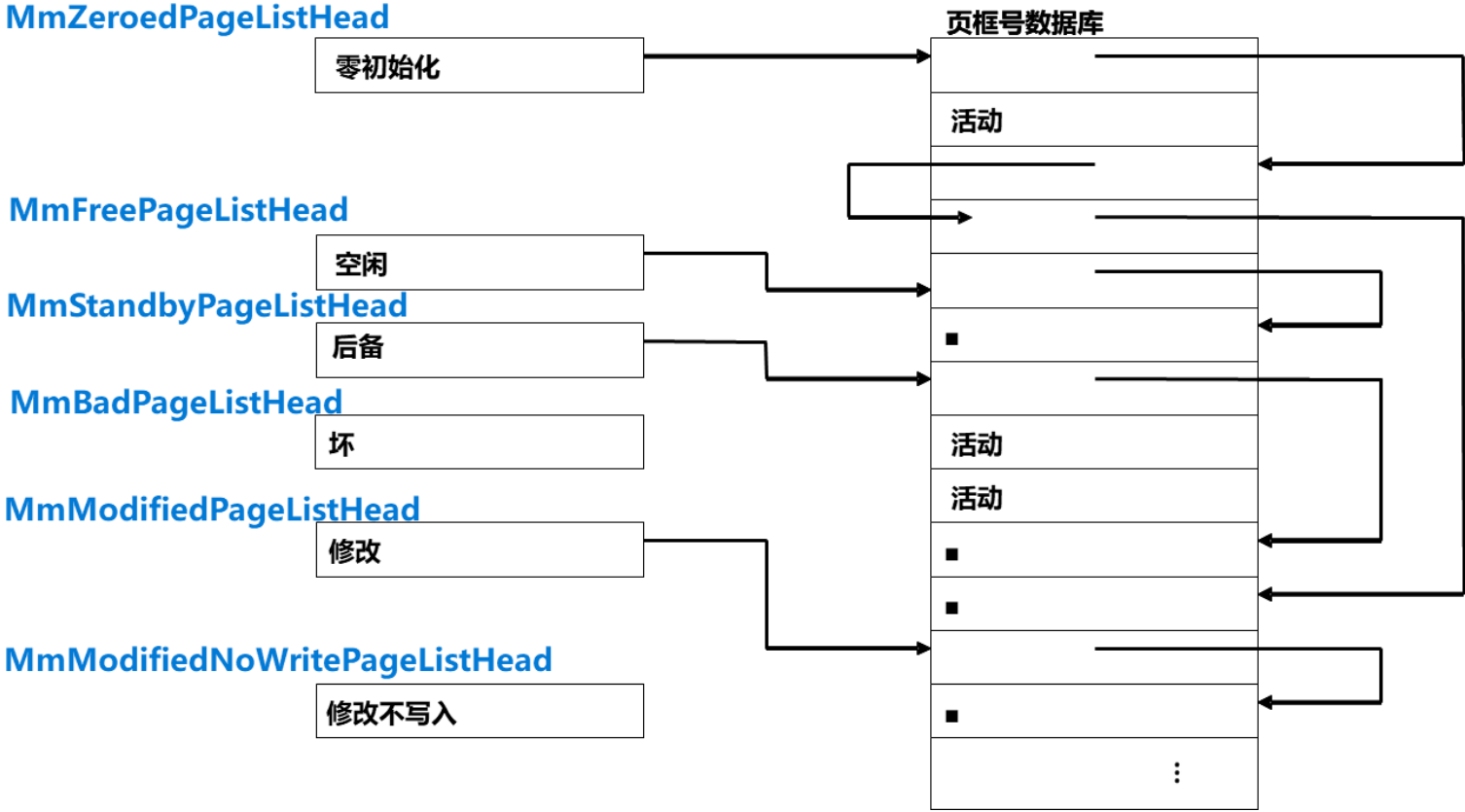
- 该页存在硬件错误，不能被使用

物理内存管理

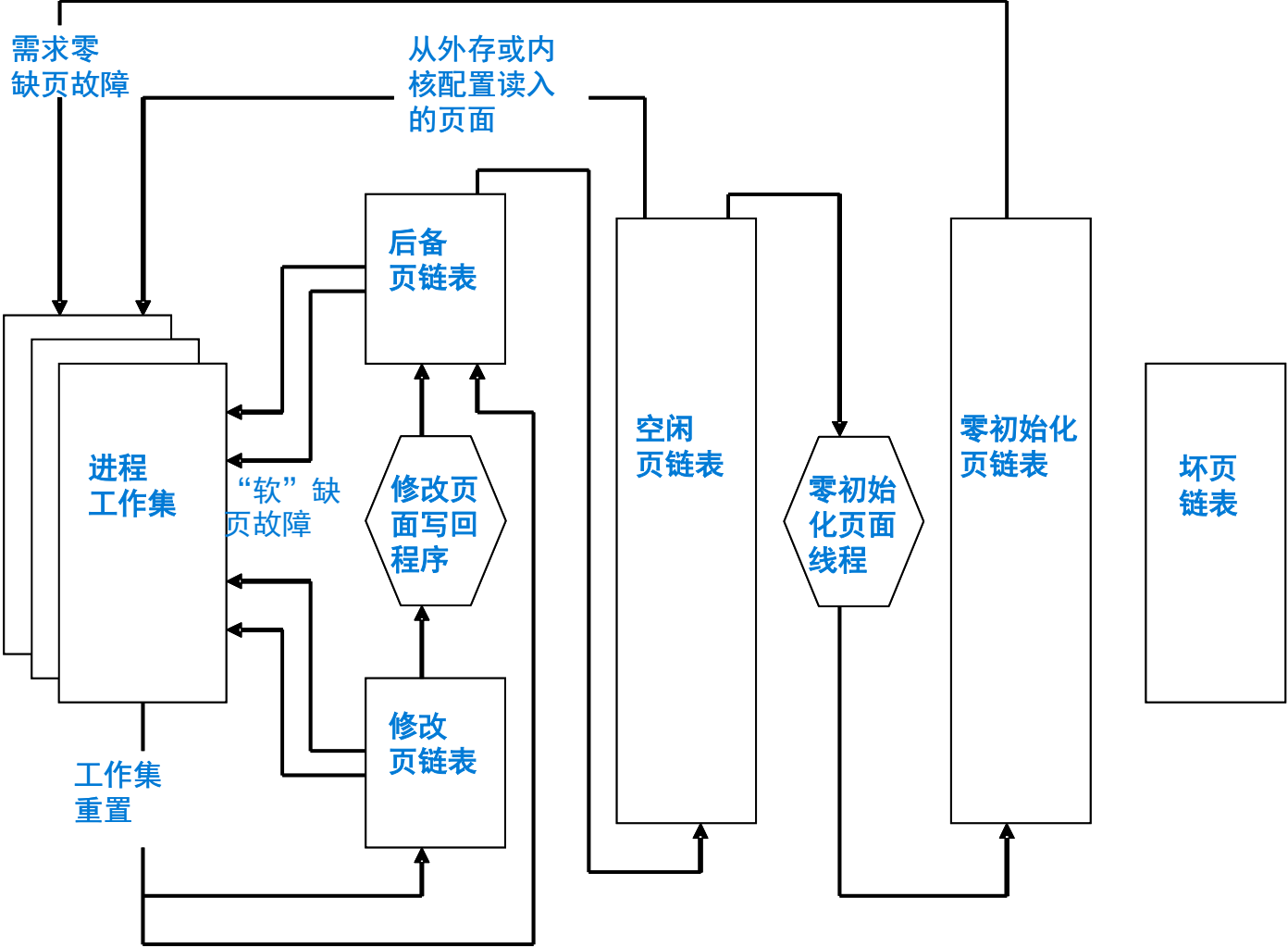


物理内存管理

■ 链表头的结构由MMPFNLIST定义



物理内存管理



大纲

01

Windows的内存管理

02

Linux的内存管理

Linux的内存管理

- 虚拟地址空间布局
- 进程用户空间的管理
- 物理内存管理
- 内存交换

- (针对64位x86-64平台)

虚拟地址空间布局

- 在64位x86-64平台上，Linux内核与Windows 11一样都依赖相同的硬件基础
 - 4级页表：PML4 → PDPT → PD → PT
 - 规范地址规则：高位必须符号扩展bit 47
- 与Windows 11的差别：
 - 虚拟地址空间的切分方式不对称
 - 内核空间的物理内存采用线性映射方式

虚拟地址空间布局

- 极端不对称
- 用户空间极大
- 内核空间压在最高地址

0xFFFFFFFF`FFFFFFFF
0xFFFFFFFF`81000000

0x00007FFF`FFFFFFFF

0x00000000`00000000

高地址区（内核态）

低地址区（用户态）

虚拟地址空间布局

- 内核空间采用线性映射(linear mapping)方式

$$\text{virtual address} = \text{physical address} + \text{offset}$$

- 线性映射仍然是通过MMU的四级页表映射，只是虚拟地址被设计成线性偏移形式

```
for phys in RAM:  
    virt = fixed_offset + phys  
    page_table[virt] = phys
```

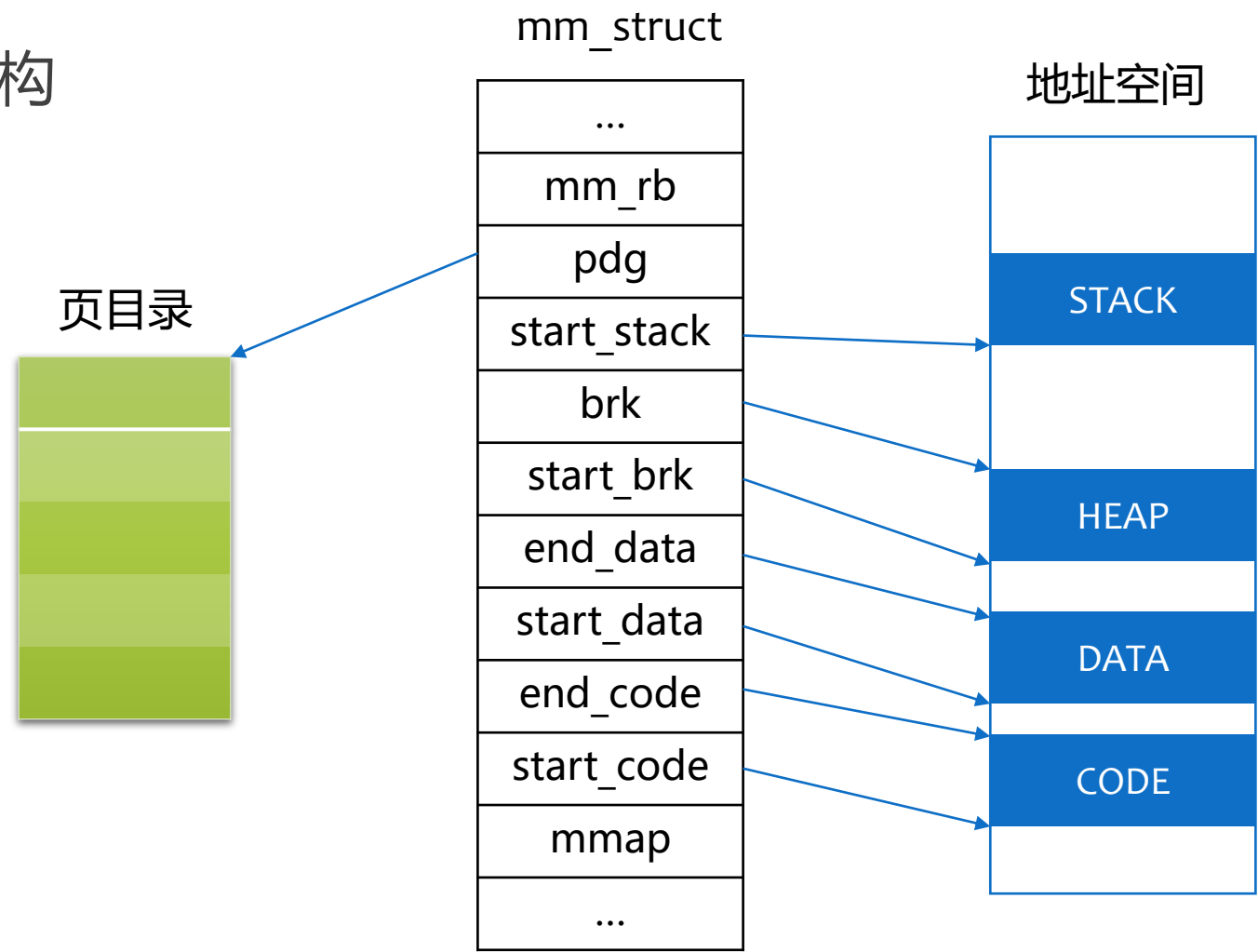

进程用户空间的管理

- 一个进程的用户空间主要由两个数据结构来描述：mm_struct和vm_area_struct
- 每个进程的虚拟用户空间由一个mm_struct结构描述，该结构中实际包含了当前执行映像的有关信息

```
struct mm_struct {
    struct vm_area_struct *mmap;           /* list of memory areas */
    struct rb_root mm_rb;                  /* red-black tree of VMAs */
    struct vm_area_struct *mmap_cache;     /* last used memory area */
    unsigned long free_area_cache;         /* 1st address space hole */
    pgd_t *pgd;                            /* page global directory */
    atomic_t mm_users;                     /* address space users */
    atomic_t mm_count;                     /* primary usage counter */
    int map_count;                          /* number of memory areas */
    struct rw_semaphore mmap_sem;          /* memory area semaphore */
    spinlock_t page_table_lock;            /* page table lock */
    struct list_head mmlist;               /* list of all mm_structs */
    unsigned long start_code;              /* start address of code */
    unsigned long end_code;                /* final address of code */
    unsigned long start_data;              /* start address of data */
    unsigned long end_data;                /* final address of data */
    unsigned long start_brk;               /* start address of heap */
    unsigned long brk;                     /* final address of heap */
    unsigned long start_stack;             /* start address of stack */
    .....
}
```

进程用户空间的管理

■ mm_struct结构



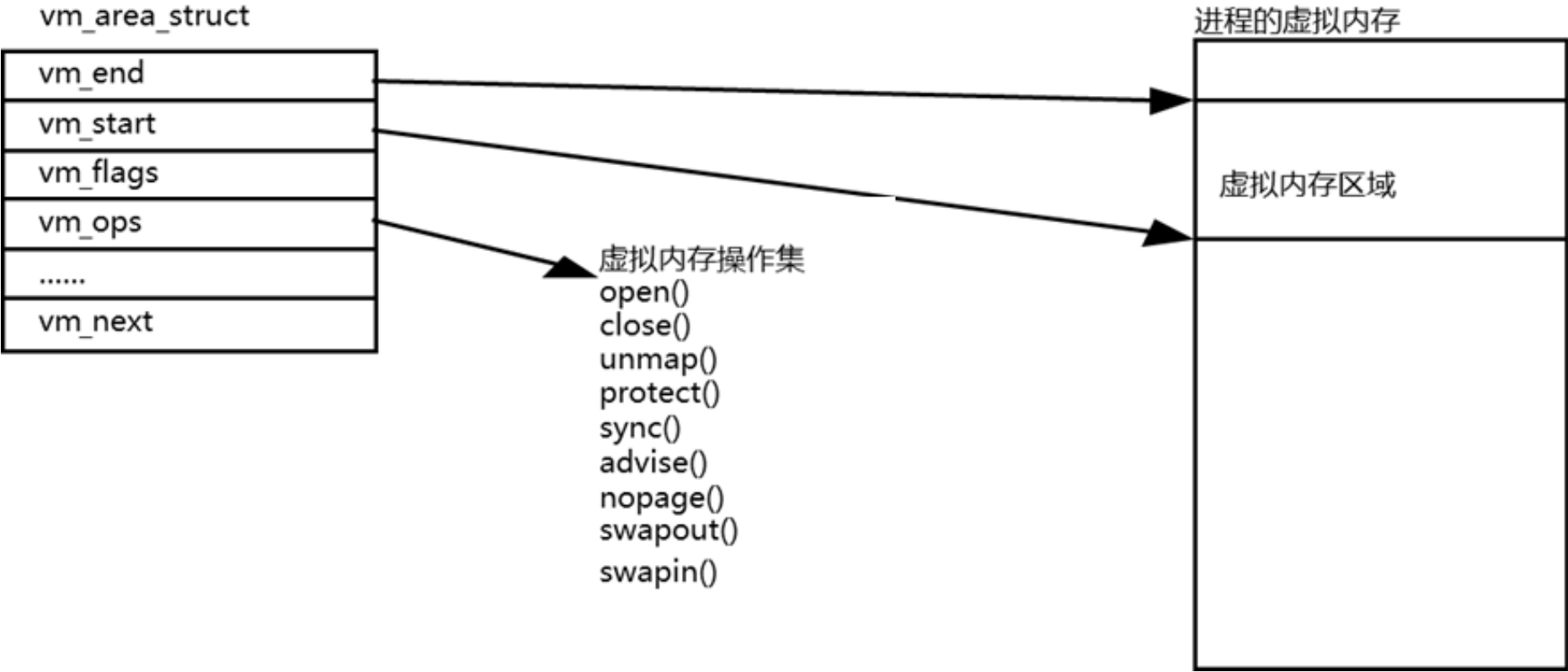
进程用户空间的管理

- vm_area_struct结构：内核每次为用户空间中分配一块内存使用时，都会生成一个vm_area_struct结构用于记录跟踪分配情况，一个vm_area_struct就代表一段虚拟内存空间

```
struct vm_area_struct {
    struct mm_struct * vm_mm;           /* The address space we belong to. */
    unsigned long vm_start;             /* Our start address within vm_mm. */
    unsigned long vm_end;
    struct vm_area_struct *vm_next;
    pgprot_t vm_page_prot;             /* Access permissions of this VMA. */
    unsigned long vm_flags;             /* Flags, see mm.h. */
    struct rb_node vm_rb;
    struct raw_prio_tree_node prio_tree_node;
    struct list_head anon_vma_node;     /* Serialized by anon_vma->lock */
    struct anon_vma *anon_vma;         /* Serialized by page_table_lock */
    struct vm_operations_struct * vm_ops;
    unsigned long vm_pgoff;
    struct file * vm_file;              /* File we map to (can be NULL). */
    void * vm_private_data;             /* was vm_pte (shared mem) */
};
```

进程用户空间的管理

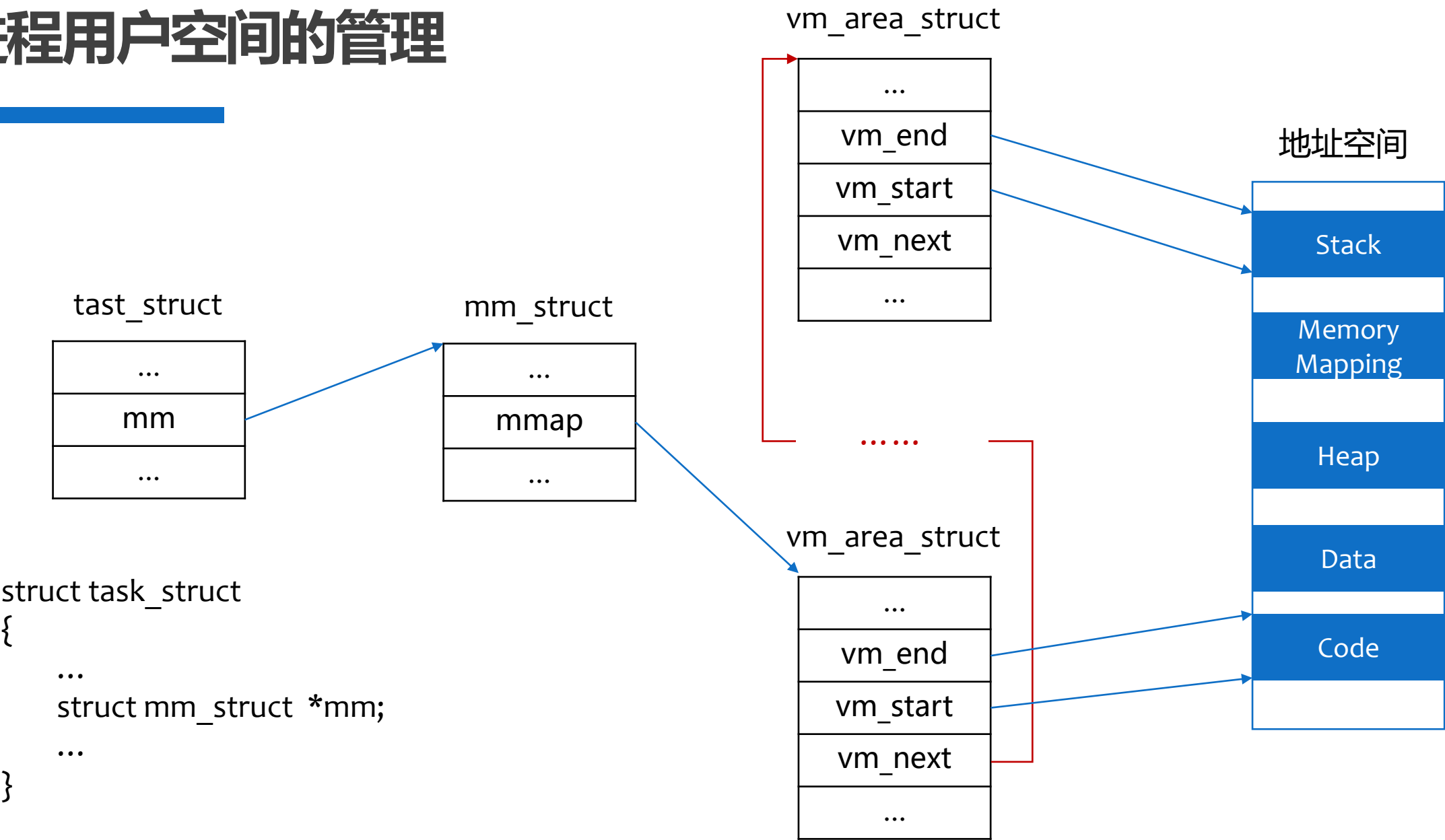
■ vm_area_struct结构



进程用户空间的管理

- 当可执行映像映射到进程的虚拟地址空间时，将产生一组vm_area_struct结构来描述虚拟内存区域的起始点和终止点，每个vm_area_struct结构代表可执行映像的一部分，可能是可执行代码，也可能是初始化的变量或未初始化的数据
- 随着vm_area_struct结构的生成，这些结构所描述的虚拟内存区域上的标准操作函数也由Linux初始化

进程用户空间的管理



进程用户空间的管理

- Linux利用vm_area_struct数据结构描述进程的虚拟内存空间，为了查找出现页故障虚拟内存相应的vm_area_struct结构的位置，Linux内核同时维护一个由vm_area_struct结构形成的AVL树(2.4.10版之前)或者红黑树(2.4.10版起)
- 利用AVL树或红黑树，可快速寻找发生页故障的虚拟地址所在的内存页区域。Linux在页故障发生时，如果搜索不到这一内存区域，则说明该虚拟地址是无效的，否则该虚拟地址是有效的

```
struct mm_struct {  
    struct vm_area_struct *mmap;           /* list of memory areas */  
    struct rb_root mm_rb;                 /* red-black tree of VMAs */  
    struct vm_area_struct *mmap_cache;     /* last used memory area */  
    unsigned long free_area_cache;        /* 1st address space hole */  
    pgd_t *pgd;                           /* page global directory */  
    .....  
};
```

进程用户空间的管理

- 内存映射：在进程的虚拟地址空间创建一个映射，有两种内存映射
 - 文件映射
 - 匿名映射
- 内存映射私有和共享两种属性

进程用户空间的管理

- 当某个程序开始运行时，可执行映像必须装入进程的虚拟地址空间。如果该程序用到了任何一个共享库，则共享库也必须装入进程的虚拟地址空间
- 实际上，Linux并不将映像装入物理内存，相反，可执行文件只是被映射到进程的虚拟地址空间中。随着程序的运行，被引用的程序部分会由操作系统装入物理内存。这种将文件映射到进程地址空间的方法称为文件映射
- 可执行文件的data段、text段是文件映射

进程用户空间的管理

- 普通的用户数据文件也可以映射到内存中
- 将文件系统中的文件映射到内存，这样用户就可以在用户空间通过 `open`、`read`、`write` 等函数操作文件

进程用户空间的管理

- 匿名映射：进程在运行时在用户空间分配内存来存储数据，这部分内存不属于任何文件，这种映射称为匿名映射
- 堆、栈、bss是匿名映射

进程用户空间的管理

- `vm_area_struct` 结构体中的 `vm_file` 成员是映射的文件，如果是匿名映射，该成员为 `NULL`

```
struct vm_area_struct {
    struct mm_struct * vm_mm;           /* The address space we belong to. */
    unsigned long vm_start;             /* Our start address within vm_mm. */
    unsigned long vm_end;
    struct vm_area_struct *vm_next;
    pgprot_t vm_page_prot;             /* Access permissions of this VMA. */
    unsigned long vm_flags;             /* Flags, see mm.h. */
    struct rb_node vm_rb;
    struct raw_prio_tree_node prio_tree_node;
    struct list_head anon_vma_node;     /* Serialized by anon_vma->lock */
    struct anon_vma *anon_vma;         /* Serialized by page_table_lock */
    struct vm_operations_struct * vm_ops;
    unsigned long vm_pgoff;
    struct file * vm_file;              /* File we map to (can be NULL). */
    void * vm_private_data;             /* was vm_pte (shared mem) */
};
```

进程用户空间的管理

- 通过一个简单的程序来观察Linux下的内存映射

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello world!\n");
```

```
    getchar();
```

```
}
```

- 观察/proc/[pid]/maps可以获得内存映射的信息
- 如果映射与文件无关，则为匿名映射

进程用户空间的管理

■与内存映射相关的系统调用

■mmap创建内存映射

`void *mmap(void *start, int length, int prot, int flags, int fd, int offset);`

■ prot——映射区域的访问模式

- PROT_READ、PROT_WRITE、PROT_EXEC

■ flags——映射类型

- MAP_SHARED、MAP_PRIVATE、MAP_ANONYMOUS

■munmap撤销内存映射

`int munmap(void *start, int length);`

进程用户空间的管理

■ 例1：创建4字节大小的匿名映射区域，父子进程共享该区域

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <sys/wait.h>

#define N 10

int main()
{
    int i, sum;
    int *result_ptr = mmap(0, 4, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, 0, 0);
    int pid = fork();
    if(pid == 0)
    {
        for(sum = 0, i = 1; i < N; i++) sum += i;
        *result_ptr = sum;
    }
    else
    {
        wait(0);
        printf("result = %d\n", *result_ptr);
    }
    munmap(result_ptr, 4);
}
```

进程用户空间的管理

■ 例2：映射一个文件到进程的虚拟地址空间

```
#include <stdio.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

main()
{
    int i, fd;
    char *buf;
    fd=open("mapfile.c", O_RDONLY);
    buf=mmap(0, 1024, PROT_READ, MAP_PRIVATE, fd, 0);
    for(i=0; i<=1024; i++)
    {
        printf("%c", buf[i]);
    }
    printf("\n");
    munmap(buf, 12);
}
```


进程用户空间的管理

- 某个可执行映像映射到进程虚拟内存中并开始执行时，因为只有很少一部分装入了物理内存，因此很快就会访问尚未装入物理内存的虚拟内存区域。这时，处理器将向 Linux 报告一个页故障及其对应的故障原因
- 页故障的出现原因有两种，一是程序出现错误，例如向随机内存地址中写入数据，这种情况下，虚拟内存可能是无效的，Linux 将向程序发送SIGSEGV信号并终止程序的运行；另一种情况是，虚拟地址有效，但其所对应的页当前不在物理内存中，这时，操作系统必须从磁盘映像或交换文件中将内存装入物理内存
- 也有可能因为进程在虚拟地址上进行的操作非法而产生页故障，例如在只读页中写入数据。这时操作系统会同样发送内存错误信号到该进程。有关页的访问控制信息(只读页、只写页、可读可写页、可执行代码页等)包含在页表项中

进程用户空间的管理

- 对有效的虚拟地址，Linux必须区分页所在的位置，即判断页是在交换文件中，还是在可执行映像中。为此，Linux通过页表项中的信息区分页所在的位置。如果该页的页表项是无效的，但非空，则说明该页处于交换文件中，操作系统要从交换文件装入页。否则，默认情况下，Linux会分配一个新的物理页并建立一个有效的页表项；对于映像的内存映射来讲，则会分配新的物理页，更新页表项属性信息，并从映像中装入页
- 这时，所需的页装入了物理内存，页表项也同时被更新，然后进程就可以继续执行了。这种只在必要时才将虚拟页装入物理内存的处理称为请求分页
- 在处理页故障的过程中，因为要涉及到磁盘访问等耗时操作，因此操作系统会选择另外一个进程进入执行状态

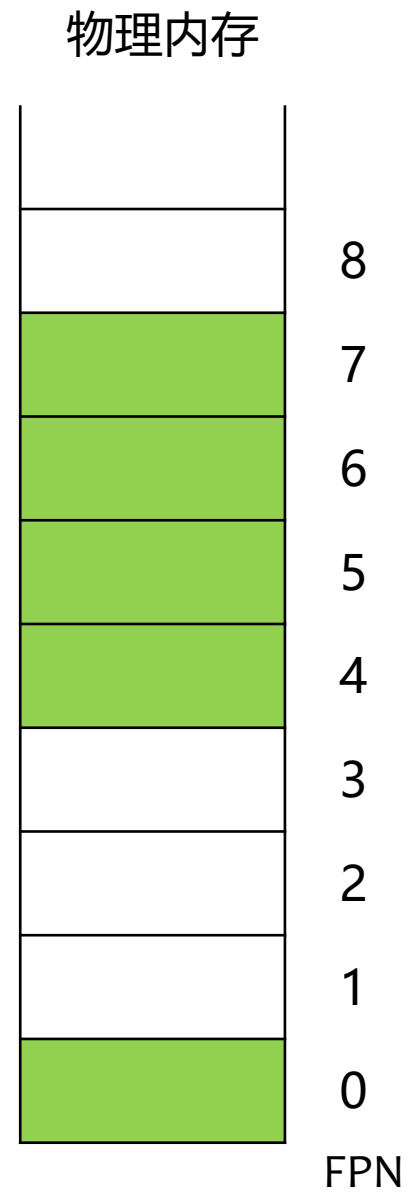
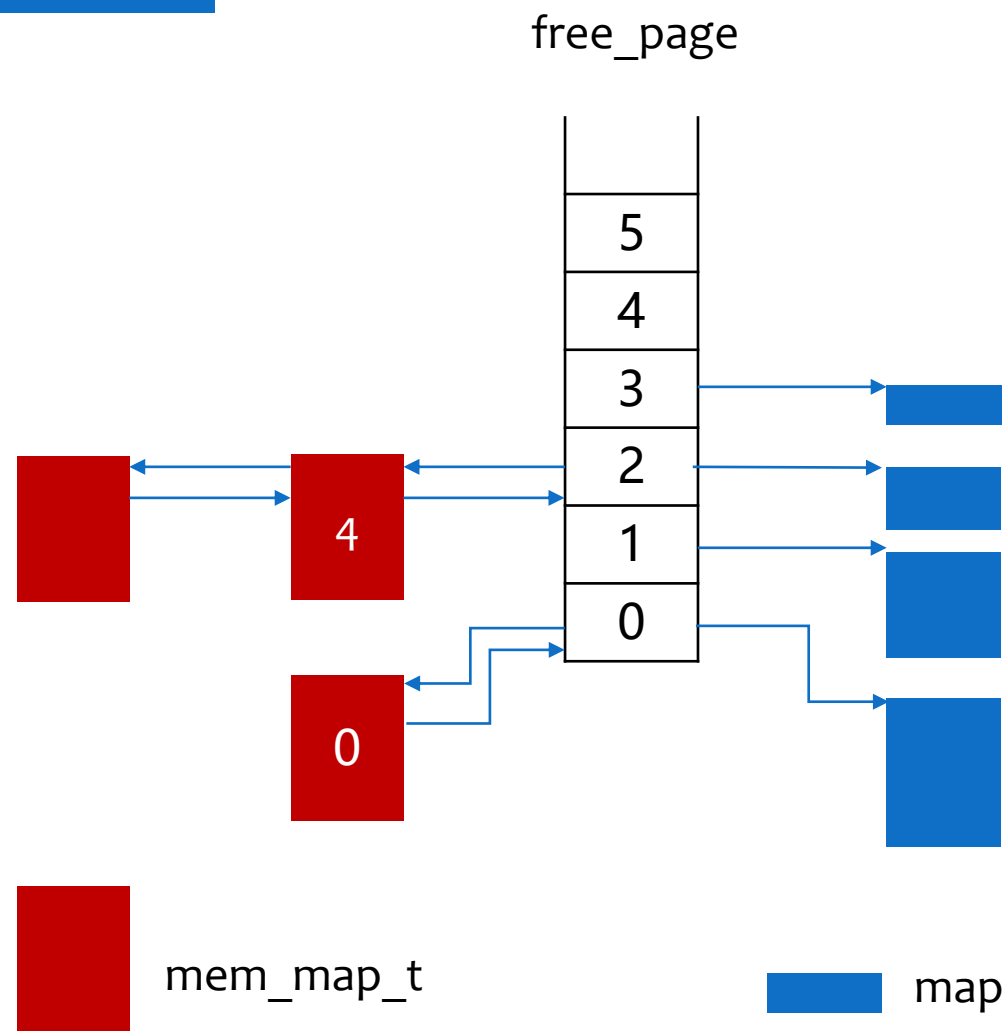
物理内存管理

- 伙伴算法(Buddy algorithm)
- 通过不断对分大的空闲内存块来获得小的空闲内存块，直到获得所需要的内存块
- 当一个空闲块被对分后，其中的一部分称为另一部分的伙伴
- 当内存块被释放时，尽可能地合并空闲块
- 空闲块的分配和合并都使用2的幂次

物理内存管理

- Linux 内核定义了一个称为 free_area的数组，该数组的每一项描述某一种页块的信息。第0个元素描述单个页的信息，第1个元素则描述以 2 个页为一个块的页块信息，第2个元素描述以4个页为一个块的页块信息，依此类推，所描述的页块大小以 2 的倍数增加
- free_area数组的每项包含两个元素：list和map。list是一个双向链表的头指针，该双向链表的每个节点包含空闲页块的起始物理页框号；而map 则是记录这种页块组分配情况的位图，例如，位图的第N位为 1，则表明第N个页块是空闲的
- 用来记录页块组分配情况的位图大小各不相同，显然页块越小，位图越大

物理内存管理



物理内存管理

- Linux采用Buddy算法有效分配和释放物理页块。按照上述数据结构，Linux可以分配的内存大小只能是1个页块，2个页块或4个页块等等。在分配物理页块时，Linux首先在free_area数组中搜索大于或等于要求尺寸的最小页块信息，然后在对应的list双向链表中寻找空闲页块，如果没有空闲页块，Linux则继续搜索更大的页块信息，直到发现一个空闲的页块为止。如果搜索到的页块大于满足要求的最小页块，则只需将该页块剩余的部分划分为小的页块并添加到相应的list链表中
- 页块的分配会导致内存的碎片化，而页块的释放则可以将页块重新组合成大的页块。如果和被释放的页块大小相等的相邻页块是空闲的，则可以将这两个页块组合成一个大的页块，这一过程一直继续到把所有可能的页块组合成尽可能大的页块为止

物理内存管理

■Linux的Slab分配器

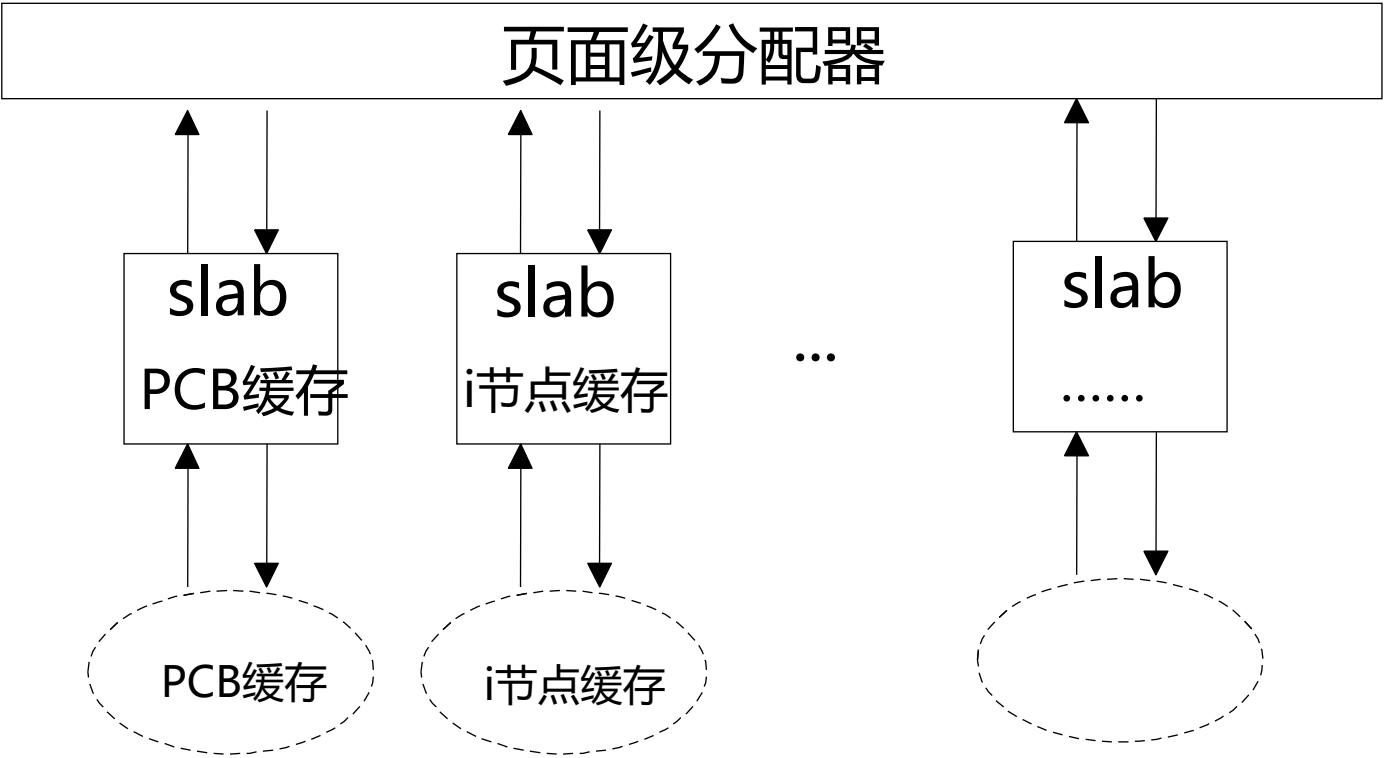
- 仅有分配页面的分配器是不能满足要求的。内核中大量使用各种数据结构，大小从几个字节到几十上百K不等，都取整到2的幂次个页面是完全不现实的
- 因此，Linux引入了slab分配模式

物理内存管理

- slab分配模式的基本思想是为经常使用的小对象建立缓存，小对象的申请和释放通过slab分配器进行
- slab分配器为不同的常用对象生成不同的缓存，并且每个缓存存储相同类型的对象
- 某种对象的缓冲区由一连串slab构成，每个slab由一个或多个页框组成，其中包含若干同种类型的对象

物理内存管理

Linux的Slab分配器



物理内存管理

■ Linux的Slab分配器

■ 主要操作有

- `kmem_cache_create/kmem_cache_destory`
- `kmem_cache_grow/kmem_cache_reap` //增长/缩减某类缓存的大小
- `kmem_cache_alloc/kmem_cache_free` //从某类缓存分配/释放一个对象
- `kmalloc/kfree` //通用缓存的分配、释放函数

■ 相关代码见slab.c

内存交换

1. 减少缓冲区和页高速缓存的大小

- 页高速缓存中包含内存映射文件的页，可能包含一些系统不再需要的页。类似地，缓冲区高速缓存中也可能包含从物理设备中读取的或写入物理设备的数据，这些缓冲区也可能不再需要，因此，这两个高速缓存可用来释放出空闲页。Linux利用Clock算法从系统中选择要丢弃的页，也即每次循环检查mem_map向量中不同的页块，象时钟的分针循环转动一样
- 每次内核的交换进程运行时，根据对物理内存的需求而选择不同页块大小的mem_map向量进行检查。如果发现某页块处于上述两个高速缓存中，则释放相应的缓冲区，并将页块重新收入free_area结构

内存交换

2. 将System V共享内存页交换出物理内存

- System V共享内存页实际是一种进程间通讯机制，系统通过将共享内存页交换到交换文件而释放物理内存
- Linux同样使用Clock算法选择要交换出物理内存的页

内存交换

3. 将页交换出物理内存或丢弃

- kswaped首先选择可交换的进程，或其中某些页可从内存中交换出或丢弃的进程。可执行映像的大部分内容可从磁盘映像中获取，因此，这些页可丢弃
- 选定要交换的进程之后，Linux将把该进程的一小部分页交换出内存，而大部分不会被交换，被锁定的页也不会被交换
- Linux利用页的寿命信息选择要交换的页，也即所谓最近最少使用 (Least Recently Used, LRU) 算法

操作系统，2026

谢谢

OPERATING SYSTEM